

“Why are we using `socket.on` inside `useEffect` here?”

“What happens if this API fails?”

Node.js Notes (Beginner Friendly)

What is Node.js?

Definition:

Node.js is a **JavaScript Runtime Environment** that allows JavaScript code to run **outside the web browser**.

- **JavaScript** = Programming language
 - **Node.js** = Software that executes JavaScript outside the browser
-

Why was Node.js created?

Originally, JavaScript could only run inside web browsers like:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Its job was mainly to make web pages interactive by working with:

- HTML
- CSS
- Buttons
- Forms
- Images
- DOM (Document Object Model)

JavaScript could **not**:

- Read files from your computer
- Create folders
- Access databases

- Run web servers
- Access the operating system

To remove these limitations, **Node.js** was created.

What is a Runtime Environment?

A runtime environment is software that executes a programming language.

Example:

- C++ → Executed by compiler + operating system
 - Python → Executed by Python Interpreter
 - Java → Executed by JVM (Java Virtual Machine)
 - JavaScript (Browser) → Executed by Browser's JavaScript Engine
 - JavaScript (Outside Browser) → Executed by Node.js
-

Browser JavaScript vs Node.js

Browser JavaScript	Node.js
Runs inside browser	Runs outside browser
Used for frontend	Used mainly for backend
Can manipulate HTML & CSS	Cannot manipulate HTML directly
Has DOM, window, document	Has file system, OS, HTTP modules
Cannot access local files directly	Can access local files
Cannot create servers	Can create servers
Cannot connect directly to databases	Can connect to databases

What Type of Code Can Node.js Run?

Node.js runs **JavaScript code**.

Example:

```
console.log("Hello World");
```

Run using:

```
node app.js
```

Output:

```
Hello World
```

What Can Node.js Do?

1. Read Files

Node.js can read files stored on your computer.

Example:

```
const fs = require("fs");  
  
const data = fs.readFileSync("hello.txt", "utf8");  
  
console.log(data);
```

2. Create Files

```
const fs = require("fs");  
  
fs.writeFileSync("test.txt", "Hello");
```

3. Delete Files

```
fs.unlinkSync("test.txt");
```

4. Create Folders

```
fs.mkdirSync("Projects");
```

5. Access Operating System Information

```
const os = require("os");  
  
console.log(os.platform());
```

Possible Output:

win32

or

linux

6. Create Web Servers

Example:

```
const http = require("http");  
  
http.createServer((req, res) => {  
  res.end("Hello");  
}).listen(3000);
```

Now open:

http://localhost:3000

Browser Output:

Hello

7. Connect to Databases

Node.js can communicate with databases like:

- MongoDB
- MySQL
- PostgreSQL

Example operations:

- Insert Data
 - Update Data
 - Delete Data
 - Fetch Data
-

8. Handle User Login

Node.js can:

- Verify Email
 - Verify Password
 - Generate JWT Token
 - Manage Sessions
-

9. Upload Files

Example:

Users upload:

resume.pdf

Node.js can:

- Receive file
 - Rename file
 - Save file
 - Upload to Cloud Storage
-

10. Build REST APIs

Example API Routes:

GET /users

POST /login

PUT /student/10

DELETE /student/5

Node.js processes these requests.

11. Build Real-Time Applications

Examples:

- WhatsApp Web
- Discord
- Slack
- Live Chat Applications

Uses technologies like **WebSockets**.

12. Automation Scripts

Examples:

- Rename thousands of files
 - Generate reports
 - Backup databases
 - Send emails automatically
-

13. Build Command-Line Tools (CLI)

Many developer tools are built using Node.js.

Examples:

- npm
- Vite
- ESLint
- Prettier

Example command:

`npm install`

14. Build Desktop Applications

Using Electron (which uses Node.js), developers build applications like:

- VS Code
 - Discord
 - Postman (earlier versions)
-

Why Can't Browser JavaScript Do These Things?

Browsers are designed with security restrictions.

Browser JavaScript **cannot** normally:

- Read your local files

- Delete files
- Create folders
- Access databases directly
- Start servers
- Access the operating system

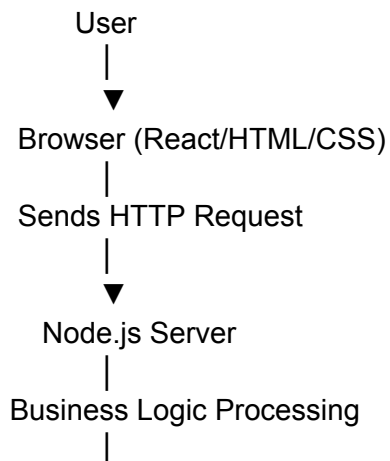
Node.js runs directly on your computer or server, so it has permission to perform these operations.

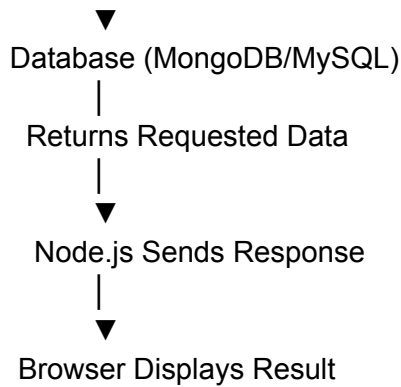
Where is Node.js Used?

Node.js is mainly used for:

- Backend Development
 - REST APIs
 - Authentication
 - Database Operations
 - File Upload Systems
 - Real-Time Chat Applications
 - Automation Scripts
 - Command-Line Tools
 - Desktop Applications
 - Microservices
-

Real-World Website Architecture





JavaScript vs Node.js

JavaScript	Node.js
Programming Language	Runtime Environment
Used for frontend and backend	Executes JavaScript outside browser
Has its own syntax	Provides APIs like <code>fs</code> , <code>http</code> , <code>path</code> , <code>os</code>
Can run in browser	Runs on computer or server

Is Node.js a Programming Language?

No.

Node.js is **not** a programming language.

It is a **JavaScript Runtime Environment**.

JavaScript is the language that Node.js executes.

Is Node.js Only Used to Run JavaScript?

No.

Running JavaScript is only the first step.

Node.js is mainly used to build:

- Backend Servers
 - REST APIs
 - Authentication Systems
 - Database Applications
 - Real-Time Chat Apps
 - Automation Scripts
 - CLI Tools
 - Desktop Applications
-

Common Node.js Modules

Module	Purpose
<code>fs</code>	File handling
<code>http</code>	Create web servers
<code>path</code>	Work with file paths
<code>os</code>	Access operating system information
<code>events</code>	Event handling
<code>url</code>	URL parsing
<code>crypto</code>	Encryption and hashing
<code>stream</code>	Handle data streams

Key Interview Points

- Node.js is a **JavaScript Runtime Environment**.
- It allows JavaScript to run **outside the browser**.

- It is built on **Google Chrome's V8 JavaScript Engine**.
 - It is mainly used for **backend development**.
 - It can create web servers, access files, connect to databases, and communicate with operating system resources.
 - It is **single-threaded** but uses an **event-driven, non-blocking I/O model**, making it efficient for handling many concurrent requests.
 - It is widely used for REST APIs, real-time applications, microservices, automation scripts, and CLI tools.
-

One-Line Summary (Exam/Interview)

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine that allows JavaScript to run outside the browser and is primarily used for building scalable backend applications, REST APIs, real-time applications, and server-side software.

Frontend => react application

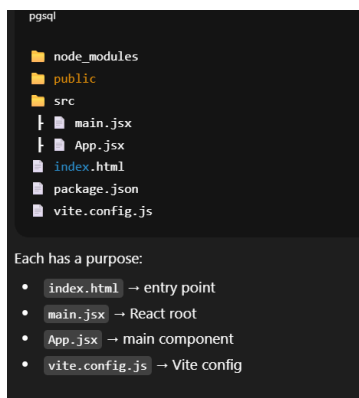
step1: `npm create vite@latest .`

1 `npm` => (Node Package Manager) Used to run scripts and install tools/packages

3 `vite@latest` => `vite` → modern frontend build tool, `@latest` → use the **latest**

stable version

-> after this inside the frontend :



Step2: `npm install`

(to get your dependencies)

Step3: `npm run dev`

(to start the frontend application)

Backend =>

Step1: `npm init -y`

(gives package.json file)

Step2: `npm i express mongoose dotenv jsonwebtoken bcrypt.js cookie-parser cloudinary socket.io`

(install all the required packages)

Step3: `npm i nodemon -D`
(dev dependencies)

Step4: in package.json , scripts will be ("dev": "nodemon index.js")

And also if we want to use import export then we have to add this
("type": "module")

Step5: create [index.js](#)

What is Express?

Short interview definition (memorize this):

Express is a minimal and flexible web framework for Node.js used to build web servers and REST APIs easily.

Why do we even need Express?

Problem with plain Node.js:

Using only Node.js, you must manually handle:

- HTTP requests
- URLs
- Request methods (GET, POST, etc.)
- Response formatting

Example (pure Node.js is messy and long).

Solution:

👉 Express simplifies all of this and gives a clean, structured way to write backend code.

Express vs Node.js (COMMON TRAP)	
Node.js	Express
Runtime environment	Framework
Low-level APIs	High-level abstraction
More code	Less code
Hard to scale	Easy to structure
🌟 Interview-ready line: Node.js provides the runtime, Express provides the structure.	

Q1: Why did you use Express?

Answer:

To create REST APIs, manage routing, middleware, authentication, and connect frontend with database efficiently.

Q2: Can Express handle real-time communication?

Answer:

No, Express handles HTTP requests. For real-time features, we use Socket.IO.

Q3: What is middleware in Express?

Answer:

Middleware is a function that runs between the incoming request and outgoing response to modify or validate data.

->The object returned by `express()` is used to configure routes, middleware, and start the HTTP server.

->Express is a function

js Copy code

```
app.METHOD(PATH, HANDLER)
```

Where:

- `METHOD` → HTTP method (`get`, `post`, `put`, `delete`)
- `PATH` → URL endpoint
- `HANDLER` → function that handles request

What is inside `req` and `res`? (INTERVIEW GOLD)

`req` contains:

- `req.body` → data sent from frontend
- `req.params` → URL parameters
- `req.query` → query string
- `req.headers` → headers
- `req.user` → added by auth middleware

`res` contains:

- `res.status(code)`
- `res.json(data)`
- `res.send()`

```
POST /users/101?sort=name&order=asc HTTP/1.1
Host: localhost:3000
Content-Type: application/json
Authorization: Bearer abc123
```

```
{
  "name": "Warsha",
  "age": 21
}
```

```

req = {
  params: {
    id: "101"
  },

  query: {
    sort: "name",
    order: "asc"
  },

  body: {
    name: "Warsha",
    age: 21
  },

  headers: {
    host: "localhost:3000",
    "content-type": "application/json",
    authorization: "Bearer abc123"
  },

  user: {
    id: 7,
    email: "warsha@gmail.com"
  }
}

```



Q: How many middleware can a route have?

A: Unlimited; they execute sequentially.

→ [index.js](#) file should be put in src folder

→ inside src create this folder:

Controllers, models, routes, lib, middleware

→

Syntax:

```
js
app.use(middlewareFunction);
```

Middleware function structure:

```
js
(req, res, next) => {
  // logic
  next();
}
```

3 Full middleware structure (VERY IMPORTANT)

```
js
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});
```

1 Global middleware (runs for ALL requests)

```
js
app.use(express.json());
```

Used for:

- JSON parsing
- Logging
- CORS

2 When a request comes (this part repeats)

Suppose request:

```
bah
GET /api/chat
```

Express does this:

```
sql
Request enters Express
↓
Runs middleware stack in order
↓
Checks routes one by one
↓
Executes the first matching route
↓
Sends response
```

Global middleware

```
js
app.use(express.json());
```

Runs for all requests.

Used for:

- Parsing JSON
- Logging
- CORS

Path-based middleware

```
js
app.use("/api", authMiddleware);
```

Runs only for routes starting with /api.

What does `app.use()` do?

Short, precise interview answer:

`app.use()` is used to register middleware in an Express application.

Middleware registered with `use` runs:

- For every HTTP method (GET, POST, etc)
- For matching paths

→ `app.use()` mounts the router at a base path.

App	Router
Main Express application	Mini application
Starts server	Cannot start server
Global scope	Feature-specific
Created using <code>express()</code>	Created using <code>express.Router()</code>
★ Strong interview line: The app handles the whole application, Router handles specific route groups.	

`express.json()` is **middleware** that converts incoming **JSON data** from the client into a JavaScript object.

Without it, Express **cannot understand JSON sent in the request body**.

What is `express.Router()` => Short, interview-ready definition: `express.Router()` creates a mini Express application that is used to **group and manage routes modularly**. Think of it as a **sub-app**.

2 Why do we need Router?

Without Router:

```
js
app.get("/api/user/login", ...)
app.get("/api/user/register", ...)
app.get("/api/chat", ...)
```

- ✗ Large file
- ✗ Hard to maintain
- ✗ No separation of concerns

With Router:

- Each feature has its own route file
- Cleaner, scalable structure

★ Interview line:

Router helps in modularizing routes and keeping the codebase clean.

→ when we do import we have to add extension as we are using module

```
JS index.js x JS auth.route.js package.json
backend > src > JS index.js > ...
1 import express from "express";
2
3 import authRoutes from "./routes/auth.route.js";
4
```

=> **authRoutes** represents an **Express**

Router object that contains a group of related routes and middleware exported from another file.

2 Main HTTP request methods (YOU MUST KNOW)

Method	Purpose	Example
GET	Fetch data	Get chat list
POST	Create data	Send message
PUT	Update data	Update profile
PATCH	Partial update	Update status
DELETE	Remove data	Delete chat

★ Interview note:

Methods follow REST principles.

4 What happens inside Express when request comes

Step-by-step:

1. Express receives request
2. Reads:
 - req.method
 - req.path
3. Checks middleware stack
4. Checks routes in order
5. First matching route executes

REST (Representational State Transfer) – Revision Notes

Definition

REST (Representational State Transfer) is an **architectural style** used to design web APIs. It defines a set of principles (constraints) that make APIs simple, scalable, and easy to use.

An API that follows these principles is called a **RESTful API**.

REST Principles (Constraints)

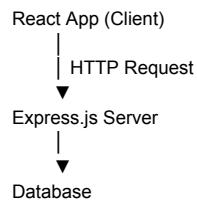
1. Client-Server Architecture

Definition

The client and server are independent and have separate responsibilities.

- **Client:** Sends requests and displays data.
- **Server:** Processes requests and manages data.

Example



Benefits

- Independent development
- Better maintainability
- Easier scalability

2. Stateless

Definition

Each request must contain **all the information** needed to process it.

The server **does not remember previous requests**.

Example

Request 1

GET /profile
Authorization: Bearer abc123

Request 2

GET /orders
Authorization: Bearer abc123

Every request sends the authentication token again.

The server never assumes:

"This is the same user from the previous request."

Benefits

- Easier to scale servers
- No session storage required
- Better reliability

3. Uniform Interface

Definition

REST APIs should have a **consistent way** of interacting with resources.

Resources are identified using URLs and manipulated using standard HTTP methods.

HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create new resource
PUT	Replace entire resource
PATCH	Update part of resource
DELETE	Delete resource

Example

GET /users
POST /users
GET /users/5
PUT /users/5
PATCH /users/5
DELETE /users/5

Benefits

- Easy to understand
- Predictable API design
- Standard across applications

4. Resource-Based URLs

Definition

Everything in REST is considered a **resource**.

Examples:

- User
- Product
- Student
- Book
- Order

Each resource has its own URL.

Good Examples

/users
/users/10

/products
/products/8

/books
/books/15

Bad Examples

/getUser
/createStudent
/deleteBook

Rule:

- Use **nouns**, not **verbs**, in URLs.

5. Representation of Resources

Definition

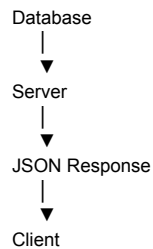
The server sends a **representation** of the resource, usually in JSON format.

Example

```
{  
  "id": 5,  
  "name": "Warsha",  
  "branch": "CSE"  
}
```

The client never accesses the database directly.

Instead:



6. Cacheable

Definition

Responses can indicate whether they may be cached.

Example

Cache-Control: max-age=3600

Meaning:

- Client can reuse the response for 1 hour.
- No need to contact the server again during that time.

Benefits

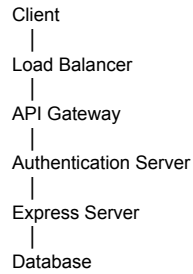
- Faster response
- Reduced server load
- Better performance

7. Layered System

Definition

A client does not need to know whether it is communicating directly with the server or through intermediate layers.

Example



The client only knows the API endpoint.

Benefits

- Better security
- Easier scaling
- Flexible architecture

REST API Example

Suppose we have a **Student Management System**.

Operation	HTTP Method	URL
Get all students	GET	<code>/students</code>
Get one student	GET	<code>/students/10</code>
Create student	POST	<code>/students</code>
Replace student	PUT	<code>/students/10</code>
Update student	PATCH	<code>/students/10</code>
Delete student	DELETE	<code>/students/10</code>

Create Student Example

Request

POST /students
Content-Type: application/json

Body

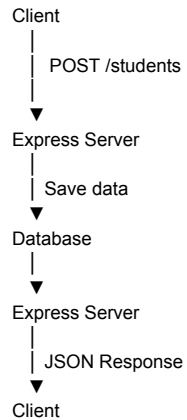
{

```
{
  "name": "Warsha",
  "branch": "CSE"
}
```

Response

```
{
  "id": 15,
  "name": "Warsha",
  "branch": "CSE"
}
```

REST API Flow



Why REST is Popular

- Simple and easy to learn
- Uses standard HTTP methods
- Platform independent
- Language independent
- Stateless (easy to scale)
- Supports JSON, XML, etc.
- Widely used in web and mobile applications

REST Principles at a Glance

Principle	Meaning
Client-Server	Client and server are independent
Stateless	Every request contains all required information
Uniform Interface	Use standard URLs and HTTP methods
Resource-Based	URLs represent resources (nouns)
Representation	Server sends resource data (usually JSON)
Cacheable	Responses may be cached
Layered System	Client doesn't know internal server architecture

Interview Definition (30-second Answer)

REST (Representational State Transfer) is an architectural style for designing web APIs. It follows principles such as **client-server architecture**, **stateless communication**, **uniform interfaces**, **resource-based URLs**, **resource representations (usually JSON)**, **cacheable responses**, and **layered systems**. These principles make APIs scalable, maintainable, and easy to use.

Quick Revision (Exam/Interview)

- REST = Representational State Transfer
- RESTful API = API following REST principles
- Resources are identified by **URLs**
- Use **nouns** in URLs (`/users`, `/products`)
- Use HTTP methods correctly:
 - GET → Read
 - POST → Create
 - PUT → Replace
 - PATCH → Partial Update
 - DELETE → Delete
- Communication is **stateless**
- Data is usually exchanged in **JSON**
- Responses can be **cached**
- Client and server are **independent**
- REST is the most commonly used architecture for modern web APIs.

→GET requests should never be used to send sensitive data.

POST supports request body

```
js
POST /api/user/login
Body: { email, password }
```

✓ Secure
✓ Standard practice

First: ONE universal rule (MEMORIZE)

All HTTP requests—regardless of method—contain:

- req.method
- req.url
- req.headers

These are ALWAYS available.

GET

Typically contains:

Source	Express
Query parameters	req.query
URL parameters	req.params
Headers	req.headers

Example:

```
http
GET /api/chat/123?limit=20
```

Extract:

```
js
req.params.id // 123
req.query.limit // 20
req.headers.authorization
```

Body: not used

POST

Typically contains:

Source	Express
Body data	req.body
Headers	req.headers
URL parameters (optional)	req.params

Example:

```
http
POST /api/message/123
Body: { "text": "Hi" }
```

Extract:

```
js
req.body.text
req.params.id
req.headers.authorization
```

PUT

Typically contains:

Source	Express
Body (full object)	req.body
URL parameters (ID)	req.params
Headers	req.headers

Example:

```
http
PUT /api/user/123
Body: { "name": "Anjali", "email": "an@x.com" }
```

Extract:

```
js
req.params.id
req.body.name
```

PATCH

Typically contains:

Source	Express
Body (partial data)	req.body
URL parameters	req.params
Headers	req.headers

Example:

```
http
PATCH /api/user/123
Body: { "status": "online" }
```

DELETE

Typically contains:

Source	Express
URL parameters	req.params
Headers	req.headers
Query params (sometimes)	req.query

Example:

```
http
DELETE /api/message/123
```

Extract:

```
js
req.params.id
```

Body: usually avoided

→under the controller we will write the function for all the routes work as in the routing file it will lead to big size.

→Database(mongodb)

→make a file .env in backend=> all the private thing like(password, token etc)

→ mogodb url, password, port,

i)**Security** – secrets not exposed in code

ii)**Flexibility** – different values for dev/prod

iii)**Config separation** – code ≠ configuration

Interview line:Environment variables help keep sensitive data out of source code.

→**dotenv.config()** reads the .env file and loads its variables into **process.env** so they can be accessed in the application.

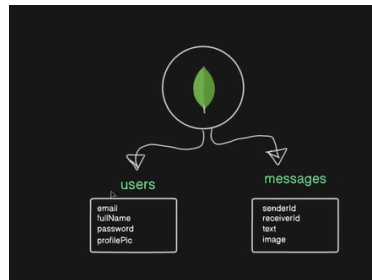
→**process.env** is a Node.js object that stores environment variables provided to the application at runtime. It allows accessing configuration values like port numbers and secrets securely.

→connecting database:

→ inside lib folder create a file [db.js](#)

→ mongoose package to connect and interact with database.

→ async function => for mongodb connection is in try catch



and also `_id` (mongodb create it)

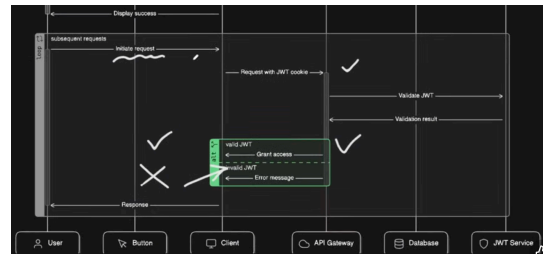
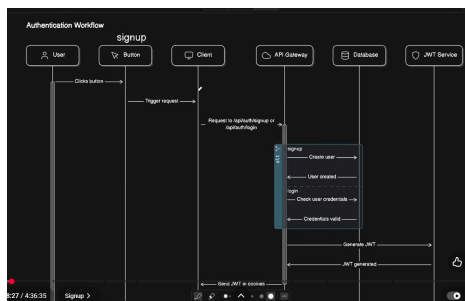
→ operation for user is `createAt` and `updateAt`

→ inside models folder there is a file [usermodel.js](#)

→ create schema

→ In MongoDB, a document is a single JSON-like record, a collection is a group of related documents, and a schema (via Mongoose) defines the structure and validation rules for those documents.

→ hashing the password in signup => `bcryptjs`, then create a token to authenticate them



→ hashing and generating token =>

→ import user model, import `bcryptjs`, then `bcrypt.genSalt(10)`, `bcrypt.hash(password, salt)`

→ **bcrypt** is a password-hashing algorithm designed to be **slow and secure** against brute-force attacks

→ A **salt** is a random string added to a password **before hashing**.

Why salt is needed (VERY IMPORTANT)

Without salt:

```
text
password123 → hash1
password123 → hash1 (same every time)
```

This allows:

- Rainbow table attacks
- Easy detection of same passwords

With salt:

```
text
password123 + saltA → hashA
password123 + saltB → hashB
```

- ✓ Same password
- ✓ Different hashes
- ✓ Much more secure

bcrypt.hash(password, salt) — WHAT IT DOES

What happens step-by-step:

1. Takes plain password
2. Combines it with the salt
3. Runs bcrypt hashing algorithm
4. Repeats hashing 2¹⁰ times (because rounds = 10)
5. Produces final hash

→`bcrypt.genSalt(10)` generates a random salt with a cost factor of 10.

`bcrypt.hash(password, salt)` combines the password and salt and applies multiple hashing rounds to produce a secure hash. This prevents brute-force and rainbow table attacks

→after creating the user hashed password and its document we will create its jwt token ,

JWT:-A JWT (JSON Web Token) is a self-contained token that proves a user's identity for subsequent requests.

The core problem JWT solves

HTTP is stateless.

That means:

- Server does not remember who you are between requests.

So after login:

❓ How does the server know future requests are from the same user?

👉 **JWT solves this problem.**

Life WITH JWT (CORRECT FLOW)

Step-by-step:

- 1. User logs in
- 2. Server verifies credentials
- 3. Server creates JWT
- 4. JWT is sent to client
- 5. Client sends JWT with every request
- 6. Server verifies JWT

JWT contains:

- User ID
- Expiry time
- Signed with secret

Client stores it (localStorage / cookie)

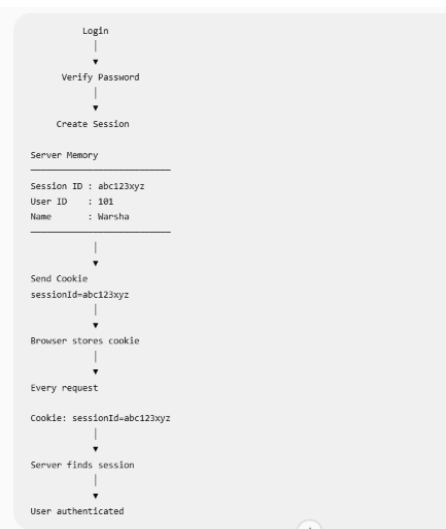
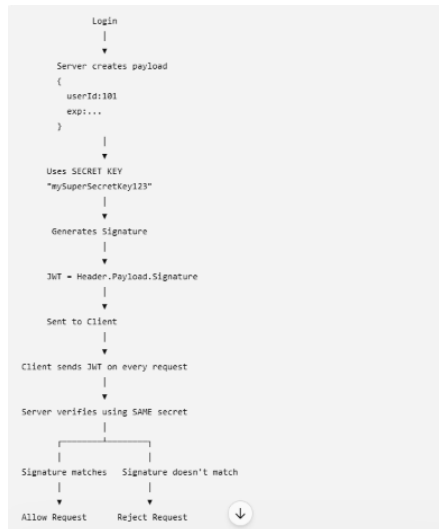
Server verifies JWT

- Valid signature?
- Not expired?

✓ Yes → allow request
✗ No → reject

Why JWT instead of server-side session? (VERY COMMON)

Session	JWT
Stored on server	Stored on client
Server keeps memory	Stateless
Harder to scale	Easy to scale
Needs session store	No DB lookup



Server-Side Session

Definition:

A stateful authentication mechanism where the server stores the user's session data, and the client stores only a Session ID (usually in a cookie).

Flow:

1. User logs in.
2. Server verifies credentials.
3. Server creates a session and generates a Session ID.
4. Session ID is sent to the client as a cookie.
5. Client sends the Session ID with every request.
6. Server retrieves the session using the Session ID and authenticates the user.

Storage:

- Server: Session data (User ID, Role, etc.)

- Client: Session ID (Cookie)

Key Points:

- Stateful authentication
- Server remembers the user
- Easy logout by deleting the session
- Common session stores: Redis, Memory, MongoDB, SQL

Best Used For:

- Traditional web applications
- Server-rendered websites

→ under the lib folder ⇒ [utils.js](#)

⇒ send jwt token in cookies

→ `res.cookie(name, value, options)`

→ `res.cookie()` is an Express response method used to **send cookies from the server to the client.**

3 How cookies work (VERY IMPORTANT FLOW)

Step-by-step:

- 1 Server sends cookie in response
- 2 Browser stores cookie
- 3 Browser automatically sends cookie with every request to same domain

```
export const generateToken = (userId, res) => {
  const token = jwt.sign({ userId }, process.env.JWT_SECRET, {
    expiresIn: "7d",
  });
  res.cookie("jwt", token, {
    maxAge: 7 * 24 * 60 * 60 * 1000, // MS
    httpOnly: true, // prevent XSS attacks cross-site scripting attacks
    sameSite: "strict", // CSRF attacks cross-site request forgery attacks
    secure: process.env.NODE_ENV !== "development",
  });
  return token;
};
```

→ now in login when we have to check if password is matching or not then we use `bcrypt.compare(password, user.password)`, if correct then create a jwt token

→ in logout we just have to clear out the cookie by deleting the jwt token

→ update profile ⇒

→ but for this we have to authenticate the user , so we use a middleware(`protectRoute`),

→ middleware(`protectRoute`) inside middleware folder in [authmiddleware.js](#)

next function use ⇒ call next thing,

→ check this token if token has been provided or not, `req.cookies.jwt`

→ **cookie-parser** is an Express middleware that **parses cookies sent by the client and makes them available in `req.cookies`.**

Where do cookies actually belong?

Cookies are:

- Defined by HTTP standard
- Sent via HTTP headers

Example:

```
http
Set-Cookie: token=abc123
Cookie: token=abc123
```

So cookies exist at the HTTP level.

What role does Node.js play?

Node.js:

- Can read raw HTTP headers
- Does NOT parse cookies automatically

```
js
req.headers.cookie
```

Node just gives you raw data.

What role does Express play?

Express:

- Built on top of Node's HTTP module
- Provides helper methods:
 - `res.cookie()` → set cookie
- But still does NOT parse cookies by default

Express makes cookies easier but doesn't fully handle them.

Where does cookie-parser fit?

cookie-parser:

- Third-party Express middleware
- Parses cookie header into `req.cookies`

So:

```
http → Node → Express → cookie-parser → req.cookies
```

Final comparison (VERY CLEAR)

Thing	Role
HTTP	Defines cookies
Browser	Stores & sends cookies
Node.js	Receives raw headers
Express	Helps set cookies
cookie-parser	Parses cookies

What does "parsing" mean?

Simple definition:

Parsing means taking raw data and converting it into a structured, usable form.

That's it.

Why parsing is needed (INTUITION)

Computers often receive data as:

- Strings
- Text
- Raw streams

But programs work better with:

- Objects
- Numbers
- Arrays
- Key-value pairs

Parsing is the conversion step.

→ Cookies are part of the HTTP protocol. Node.js receives them as raw headers, Express provides utilities to set them, and middleware like cookie-parser parses them into usable objects.

→ jwt valid or not = >we do it by identifying userid as during creation of jwt token, we have put userid there,

What does jwt.verify() do? (CORE IDEA)

Interview-ready definition:

`jwt.verify()` checks whether a JWT is valid, untampered, and not expired, and then decodes its payload.

So it does THREE things:

1. Verify signature
2. Check expiry
3. Decode payload

How signature is created (internally)

When the server creates a token:

```
js
jwt.sign(payload, secret)
```

JWT has 3 parts:

```
js
header → payload → signature
```

The signature is created like this:

```
js
const signature = HMAC(
  base64urlEncode(header) + "." + base64urlEncode(payload),
  secretKey
)
```

Key points:

- Uses a secret key (only server knows)
- Uses a hashing algorithm (e.g. HS256)
- Client cannot recreate this signature

→ In JWT, the signature ensures the token's integrity and authenticity, while the payload contains claims such as the user ID and expiration time. The payload is readable but protected from tampering by the signature.

→ now for updating profile after authenticate we have to add cloudinary to use it as storing images and videos

→ cloud_name, cloud_api_key, cloud_api_secret is stored in .env file.

→ to configure cloudinary we do it in lib/cloudinary.js, (cloudinary (the package) exports multiple versions of its API)

→ v2 from cloudinary and config from dotenv

→ v2 is an object exported by the Cloudinary library. And v2 is the version-2

API object

→ In Cloudinary, v2 is an object that exposes the version-2 API methods for uploading and managing media. In dotenv, config is a function that loads environment variables from a .env file into process.env.

→ cloudinary.config() initializes the Cloudinary SDK with account credentials. It stores these values internally so that all subsequent upload or API calls are authenticated automatically.

What does cloudinary.config() do?

It gives you access to Cloudinary features like:

```
cloudinary.config({
  cloud_name: 'your-cloud-name',
  api_key: 'your-api-key',
  api_secret: 'your-api-secret'
});
```

Or manually:

```
cloudinary.v2.config({
  cloud_name: 'your-cloud-name',
  api_key: 'your-api-key',
  api_secret: 'your-api-secret'
});
```

Why use v2 specifically?

Because:

- Cloudinary v2 API is easier
- You probably don't need it
- Not better than v1

It is the recommended Cloudinary SDK object that provides optimal and managed environment.

```
js
v2 === { config, uploader, api, url, ... }
```

config from dotenv

Typical code

```
js
const dotenv = require('dotenv');
dotenv.config();
```

or

```
js
import { config } from 'dotenv';
config();
```

What is config?

config is a function exported by the dotenv package.

Function

Not object

Not array

What does config() do?

When you call:

```
js
config();
```

It:

1. Reads .env file
2. Parses key-value pairs
3. Adds them into process.env

Example:

```
env
PORT=5000
JWT_SECRET=abc
```

After:

```
js
config();
process.env.PORT; // "5000"
```

```
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});
```

Full flow (VERY IMPORTANT)

```
scss
.env file
↓
dotenv.config()
↓
process.env values available
↓
cloudinary.config({ credentials })
↓
cloudinary.uploader.upload() works
```

→ {new:true} => after update give the document of that user

→ now make message schema

```
const messageSchema = new mongoose.Schema(
  {
    senderId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    receiverId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    text: {
      type: String,
    },
    image: {
      type: String,
    },
  }
);
```

Frontend:-

```
npm i react-router-dom react-hot-toast
```

```
burakorkmez@Buraks-MacBook-Pro frontend % npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p
```

App.jsx

```
NPM PNPM Yarn Bun  
npm i -D daisyui@latest
```

Daisyui

- start application => npm run dev
- browserouter component is used for

What problem does it solve?

In a normal website:

- Clicking a link → page reload
- Server returns a new HTML page

In a React SPA:

- We want URL changes without page reload
- We want different components for different URLs

👉 `BrowserRouter` enables this.

What exactly does `BrowserRouter` do?

Internally, it:

1. Uses the browser's history API (`pushState` , `replaceState`)
2. Watches the URL
3. Decides which React component to render
4. Prevents full page reloads

When URL changes from:

```
bash  
/login → /chat
```

What happens:

- Browser URL changes
- No HTTP request to server
- React Router renders `<chat />`

BrowserRouter vs normal anchor (`<a>`)

❌ `<a href="/chat">`

- Reloads page
- React state lost

✅ `<Link to="/chat">`

- Uses `BrowserRouter`
- No reload

- under src folder there is components(navbar.jsx) and pages folder

Definition (INTERVIEW-READY):

`Route` defines a mapping between a URL path and a React component.

It answers:

"When the URL is this, render that component."

Example

```
js  
<Route path="/login" element=<Login /> />
```

Meaning:

```
js  
URL = /login → render <Login />
```

What is `Routes`?

Definition (INTERVIEW-READY):

`Routes` is a container component that holds multiple `Route` components and renders the best matching one.

Example

```
js  
<Routes>  
  <Route path="/" element=<Home /> />  
  <Route path="/login" element=<Login /> />  
  <Route path="/chat/:id" element=<Chat /> />  
</Routes>
```

- In React Router, **Route** defines which component should render for a given path, while **Routes** acts as a container that evaluates and renders the matching route based on the current URL.

```
burakorkmez@Buraks-MacBook-Pro frontend % npm i axios zustand
```

What is Axios?

Short, interview-ready definition:
Axios is a JavaScript library used to make HTTP requests from the browser or Node.js.

What problem does Axios solve?
You need to:

- Call backend APIs
- Send data (login, messages)
- Receive responses (chats, users)

Axios makes this simple and consistent.

Example usage

```
js
import axios from 'axios';

const res = await axios.post('/api/auth/login', {
  email,
  password
});
```

What Axios provides (IMPORTANT)

- Automatic JSON parsing
- Supports GET, POST, PUT, DELETE
- Request & response interceptors
- Handles headers (JWT, cookies)
- Better error handling than `fetch`

★ Interview line:
Axios simplifies API calls and provides interceptors for authentication.

What is Zustand?

Short, interview-ready definition:
Zustand is a lightweight state management library for React.

Why Zustand is needed?

React state problems:

- Props drilling
- Global state lib:
 - Logged-in user
 - Chats
 - Selected chat
 - Notifications

Zustand solves this without Redux complexity.

Basic Zustand store example

```
js
import { create } from 'zustand';

const useAuthStore = create((set) => ({
  user: null,
  setUser: (user) => set({ user })
}));

Use anywhere:

js
const user = useAuthStore((state) => state.user);
```

Why Zustand over Redux? (INTERVIEW)

Redux	Zustand
Boilerplate	Minimal
Actions + reducers	Direct state updates
Harder to learn	Easy
Heavy	Lightweight

★ Interview line:
Zustand offers global state management with minimal boilerplate.

How Axios and Zustand work together (IMPORTANT)

In your chat app:

- Axios → fetch data from backend
- Zustand → store that data globally

-> we initialize the axios under the `src/lib/axios.js` file
Instances for the entire project = >

What is this code?

```
js
export const axiosInstance = axios.create({
  baseURL: 'http://localhost:5001/api',
  withCredentials: true,
});
```

Short, interview-ready definition:
This creates a custom Axios instance with predefined configuration that will be reused for all API requests.

Why do we create an Axios instance?

Without it ❌:

```
js
axios.get('http://localhost:5001/api/chat', { withCredentials: true });
axios.post('http://localhost:5001/api/login', { withCredentials: true });
```

Problems:

- Repetition
- Easy to forget config
- Hard to maintain

With instance ✅:

```
js
axiosInstance.get('/chat');
axiosInstance.post('/login');
```

★ Interview line:
Axios instances help centralize configuration and reduce duplication.

What does each option mean?

baseURL

```
js
baseURL: 'http://localhost:5001/api'
```

Meaning:

- This prefix is automatically added to every request

So:

```
js
axiosInstance.get('/chat');
```

Actually sends request to:

withCredentials: true

```
js
withCredentials: true
```

Meaning:
Allow cookies to be sent along with requests.

This is required when:

- Auth is cookie-based (JWT in cookies)
- Frontend and backend are on different origins

Without it:

- ❌ Cookies not sent
- ❌ Auth fails

→ `axios.create()` is used to create a reusable Axios instance with common configuration like base URL and credentials, making API calls cleaner and consistent across the application.
→ `zustand` -> global state management library => instead of sending to each page we create it at one place and use it everywhere

- create store folder inside src folder, and create a file [useAuthStore.js](#) file
 -> here we can have bunch of states and functions that can be use in different components.

```
import { create } from "zustand";

export const useAuthStore = create((set) => ({
  authUser: null,
  isCheckingAuth: true,
}));
```

This code creates a **global authentication state store** using Zustand, which can be accessed from **any React component**.

Why Zustand instead of React Context?

Problem with Context:

- Boilerplate
- Extra providers
- Performance issues

Zustand advantages:

- No provider needed
- Simple API
- Less re-rendering

Interview line:

Zustand simplifies global state management with minimal boilerplate.

A global store is a centralized place to store and manage state that needs to be shared across multiple components.

Why do we need a global store?

Problem without it:

```

import {
  Navbar,
  Sidebar,
  Chat,
} from './components';

export const useAuthStore = create((set) => ({
  authUser: null,
  isCheckingAuth: true,
}));
```

All need:

- Logged-in user
- Auth status

Without global store:

- ✗ Props drilling
- ✗ Complex Context

With global store:

- ✓ Single source of truth

- **State** is data that can change over time and affects what is rendered on the UI.
 → A **hook** is a special function in React that lets you “hook into” React features like state, lifecycle, or external stores.

→ now we are going to work on the authentication in web application like while application checking the authentication it will do spinning . and using axiosinstance here to send request to endpoint to check for authentication .and storing the response. And according to it we will set the authuser value and ischeckauth value

→ A **hook** is a special function in React that lets function components use React features like state, lifecycle, and side effects.

- **Mounting** → Component is created and added to the DOM
Updating → Component re-renders when state/props change
Unmounting → Component is removed from the DOM

Why hooks exist

Before hooks:

- Only class components had:
 - State
 - Lifecycle methods

Hooks allow:

- Function components to do everything classes could do

Rules of hooks (VERY IMPORTANT)

- Hooks start with `use`
- Call hooks only:
 - At the top level
 - Inside React function components or custom hooks

Why Hooks Were Introduced

Before hooks:

- State and lifecycle methods were available **only in class components**
- Logic reuse was difficult (HOCs, render props were complex)

With hooks:

- You can write **clean, readable functional components**
- Logic can be **reused easily**
- No need for classes

Commonly Used React Hooks	
Hook	Purpose
<code>useState</code>	Manage state in function components
<code>useEffect</code>	Handle side effects (API calls, timers, subscriptions)
<code>useContext</code>	Access global data without prop drilling
<code>useRef</code>	Access DOM elements or persist values
<code>useMemo</code>	Optimize expensive calculations
<code>useCallback</code>	Optimize function references
<code>useReducer</code>	Manage complex state logic

What is a Side Effect?

A side effect is any operation that affects something outside the component's rendering process.

- Rendering should be pure
- Side effects are impure actions

Common Side Effects in React

- API calls (`fetch`, `axios`)
- Timers (`setInterval`, `setInterval`)
- Subscriptions (Websocket, events)
- DOM manipulation
- Logging
- LocalStorage / sessionStorage

Step-by-step execution

- Initial render**
 - React renders `<App />`
 - `authUser` is initially `null`
 - `isCheckingAuth` is probably `true`
- After render** → `useEffect` runs


```

      useEffect(() => {
        checkAuth();
      }, []);
      
```

This usually:

- Calls backend (`/auth/me`)
- Verifies JWT (cookie)
- Updates global store:
 - `authUser = user` OR `null`
 - `isCheckingAuth = false`

→ **CORS (Cross-Origin Resource Sharing)** is a browser security mechanism that controls whether a web page can request resources from a different origin.

What is an "origin"?

An origin is defined by three things:

protocol + domain + port

Why does CORS exist?

Without CORS:

- Any website could read data from any other website
- Huge security risk (stealing cookies, data)

Interview line:

CORS protects users from malicious cross-site requests.

What is a CORS error?

Typical error:

"Blocked by CORS policy."

This means:

- Browser blocked the request
- Backend did not explicitly allow this origin

Important:

CORS errors are browser-enforced, not server errors.

Who causes the CORS error?

- Backend
- Axios
- React
- Browser

The browser checks response headers and decides.

What headers are involved in CORS?

Key response headers:

```

Access-Control-Allow-Origin
Access-Control-Allow-Credentials
Access-Control-Allow-Methods
Access-Control-Allow-Headers
  
```

If these are missing or wrong → browser blocks response.

→ A CORS error occurs when a browser blocks a cross-origin request because the server has not allowed that origin. CORS is a **browser security mechanism** that prevents unauthorized cross-site access.

→ in backend folder in [index.js](#)

→ `npm i cors`

```

app.use(cors({
  origin: "http://localhost:5173",
  credentials: true
}));
  
```

→ loader component is used for loading which is imported from lucid-react package

→ `react-hot-toast`

→ **react-hot-toast** is a lightweight React library used to show **non-blocking toast notifications** (small popup messages).

→ we used it for error message.

→ for the correct format, regular expression is used.

→ password length ≥ 6

→ if it successfully then we send the formdata to the signupauth function.

→ `useRef` is used in messageinput

What is useState?

Definition (INTERVIEW-READY):
 useState is a React hook used to store stateful data that affects rendering.

Example

```

  import { useState } from 'react';

  const [count, setCount] = useState(0);

  // count - state value
  // setCount - updates state
  // Updating state -> component re-renders
  
```

Key points about useState

- Triggers re-render when updated
- Used for UI-related data
- Asynchronous update

Examples:

- Form input values
- Toggle states
- Logged-in user (local)
- Counters

What is useRef?

Definition (INTERVIEW-READY):
 useRef is a React hook used to store a mutable value that does NOT cause re-render when changed.

Example

```

  import { useRef } from 'react';

  const inputRef = useRef(null);

  // Access value:
  // inputRef.current
  
```

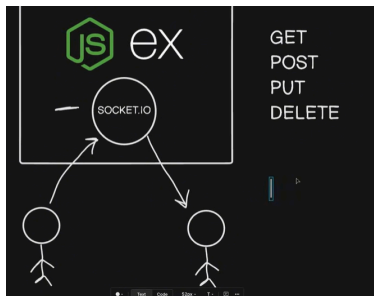
Key points about useRef

- Value persists across renders
- Updating .current **does NOT** re-render
- Commonly used to access DOM elements

→ to use the variable inside the useauthuser we use get

→ chat-start from other side and chat-end from my side.

→ now we will add socket.io to make my application chat work in real time (without refreshing it



get updated) => real time communication



→ both client and server will use [socket.on\(\)](https://socket.io/docs/v4/socket.on/) =>

→ in backend/src/lib/[socket.js](https://socket.io/docs/v4/socket.js)

```

import { Server } from "socket.io";
import http from "http";
import express from "express";

const app = express();
const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: ["http://localhost:5173"]
  }
});

export { io };
  
```

```

login: async (data) => {
  set({ isLoggingIn: true });
  try {
    const res = await axiosInstance.post("/auth/login", data);
    set({ authUser: res.data });
    toast.success("Logged in successfully");
  } catch (error) {
    // handle error
  }
  get().connectSocket();
}
  
```

```

connectSocket: () => {
  const socket = io("http://localhost:5001");
},
disconnectSocket: () => {}
  
```

```
import http from "http";
import express from "express";

const app = express();
const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: ["http://localhost:5173"],
  },
});

// used to store online users
const userSocketMap = {}; // {userId: socketId}

io.on("connection", (socket) => {
  console.log("A user connected", socket.id);

  const userId = socket.handshake.query.userId
  socket.on("disconnect", () => {
    console.log("A user disconnected", socket.id);
  });
});
```

```
const app = express();
const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: ["http://localhost:5173"],
  },
});

// used to store online users
const userSocketMap = {}; // {userId: socketId}

io.on("connection", (socket) => {
  console.log("A user connected", socket.id);

  const userId = socket.handshake.query.userId
  if(userId) userSocketMap[userId] = socket.id

  // io.emit()
  io.emit()

  socket.on("disconnect", () => {
    console.log("A user disconnected", socket.id);
  });
});
```

```
// used to store online users
const userSocketMap = {}; // {userId: socketId}

io.on("connection", (socket) => {
  console.log("A user connected", socket.id);

  const userId = socket.handshake.query.userId;
  if (userId) userSocketMap[userId] = socket.id;

  // io.emit() is used to send events to all the connected clients
  io.emit("getOnlineUsers", Object.keys(userSocketMap));

  socket.on("disconnect", () => {
    console.log("A user disconnected", socket.id);
    delete userSocketMap[userId];
    io.emit("getOnlineUsers", Object.keys(userSocketMap));
  });
});

export { io, app, server };
```

```
subscribeToMessages: () => {
  const { selectedUser } = get();
  if (!selectedUser) return;

  const socket = useAuthStore.getState().socket;

  // todo: optimize this one later
  socket.on("newMessage", (newMessage) => {
    set({
      messages: [...get().messages, newMessage],
    });
  });
},

unsubscribeFromMessages: () => {
  const socket = useAuthStore.getState().socket;
  socket.off("newMessage");
},
```

Why WebSocket is Needed

Problem with HTTP

- HTTP is request-response based
- Server cannot push data unless client asks
- Not efficient for real-time apps

Solution: WebSocket

- Single connection
- Two-way communication
- Low latency
- Server can push data anytime

How WebSocket Works (Flow)

1. Client sends an HTTP request with Upgrade: websocket
2. Server accepts the upgrade
3. Connection switches from HTTP → WebSocket
4. Data flows both ways continuously

arduino

Client → HTTP Handshake → Server

Client ← Switch to WS → Server

Client ↔ Real-time Data ↔ Server

WebSocket is a communication protocol that provides a full-duplex, persistent connection between a **client** (browser/app) and a **server**.

2 SOCKET.IO SERVER SETUP

```
js
const io = new Server(server, {
  cors: {
    origin: ["http://localhost:5173"],
  },
});
```

What this does:

- Creates a **WebSocket** server
- Allows frontend (5173) to connect

So now:

- REST APIs → Express
- Real-time events → Socket.IO

```
io.on("connection", ...)
```

means:

"When a client connects to my WebSocket server, run this code."

What is socket?

```
js
(socket) => { ... }
```

- `socket` represents **ONE** connected client
- Each client gets its own socket object

What `socket` contains

- `socket.id` → unique ID for that client
- `socket.emit()` → send message to that client
- `socket.on()` → listen to messages from that client
- `socket.disconnect()` → close connection

★ **Important**

- `io` = all clients
- `socket` = one specific client

It listens for new client connections on a Socket.IO server and creates a socket object for each connected client.

`const userId = socket.handshake.query.userId; =>` server know which userid is this through socket

REAL-TIME DELIVERY (SOCKET MAGIC)

```
js
const receiverSocketId = getReceiverSocketId(receiverId);
```

- Check if receiver is online

```
js
if (receiverSocketId) {
  io.to(receiverSocketId).emit("newMessage", newMessage);
}
```

- ✓ If online → send instantly
- ✗ If offline → message will load later from DB

★ **Interview GOLD:**

SocketIO is used for real-time delivery, REST API for persistence.

COMPLETE FLOW (LOCK THIS IN)

```
graph TD
    User[User sends message] --> DB[DB: findMessageById]
    DB --> User
    User --> Server[Send message to io]
    Server --> Receiver[Check if receiver online]
    Receiver --> Socket[If yes = socket emit]
    Socket --> Receiver[If no = receiver gets message later]
```

🔑 **INTERVIEW-READY SUMMARY (VERY IMPORTANT)**

We use Express API to store messages in the database and Socket.IO to deliver messages in real time. A user to socket mapping allows targeted message delivery.

Create socket client

```
js
const socket = io(BASE_URL, {
  query: {
    userId: authUser._id,
  },
});
```

This sends:

```
graphql
userId → backend socket.handshake.query.userId
```

Listen for online users

```
js
socket.on("getOnlineUsers", (userIds) => {
  res({ onlineUsers: userIds });
});
```

Backend emits:

```
js
io.emit("getOnlineUsers", Object.keys(userSocketMap));
```

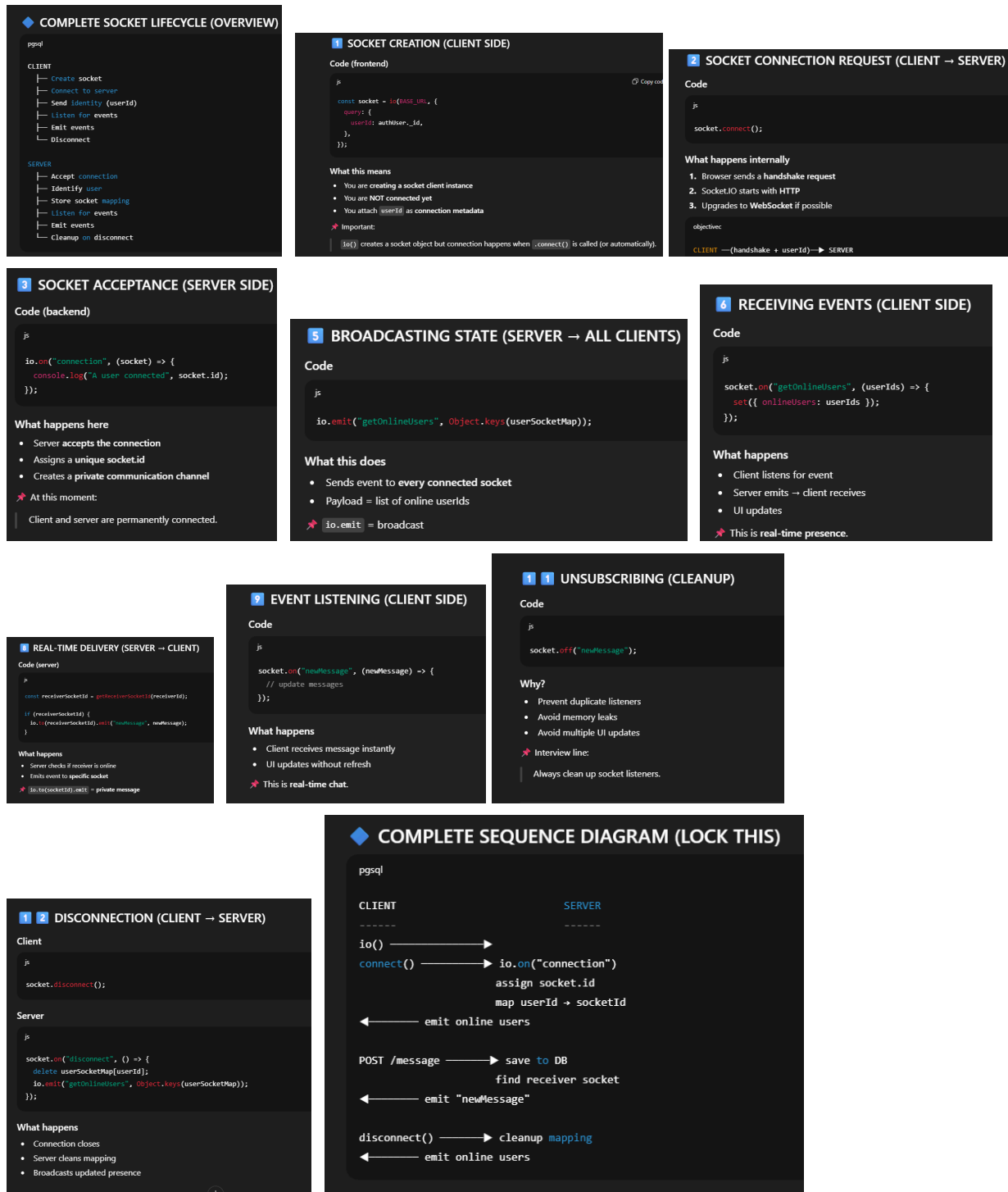
Frontend updates:

- Online badges
- Green dots
- Presence UI

```
graph TD
    User[User logs in] --> Auth[Auth store connects socket]
    Auth --> Backend[Backend stores socketId]
    Backend --> User
    User --> Chat[User opens chat]
    Chat --> Store[Chat store subscribes to "newMessage"]
    Store --> Backend
    Backend --> Emit[Backend emits message]
    Emit --> UI[Frontend filters & updates UI]
```

The frontend uses Zustand to manage authentication and socket state. Socket connections are established after login, and chat components subscribe to message events dynamically. Messages are filtered by selected user to ensure correct real-time updates.

Socket.IO creates a **persistent, bidirectional connection** between client and server so they can send and receive events in real time.



The client creates a socket and connects to the server with user identity. The server binds the user to a socket ID and uses it to send targeted real-time events. Messages are stored via REST APIs and delivered instantly using [Socket.IO](#).

Stateless means **not storing any information (state)** about previous interactions.

Persistence means the **ability to store data so that it is not lost**, even after:

- the program stops running
- the server restarts
- the system is turned off

Global state management is a way to **store and manage data that needs to be shared across many components** in an application, **without passing props manually at every level**.

Middleware is a function that executes between the incoming request and outgoing response to perform operations like authentication, logging, or validation